

Clase 153 — Análisis de malware en scripts: PowerShell y JavaScript

Parte: 6 — Análisis de malware · Fuente: *Learning Malware Analysis* (Monnappa) y MITRE ATT&CK 🕒

Duración estimada: 110 min · Nivel: Intermedio

Objetivo

Desofuscar y entender malware basado en scripts, hoy dominante en las primeras etapas de intrusión: PowerShell, JScript/JavaScript (WSH), HTA y VBScript. El alumno aprenderá a deshacer capas de ofuscación, reconocer patrones de descarga e inyección en memoria, y usar el logging del propio sistema para reconstruir lo que el script hizo.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Reconocer** patrones de ofuscación en PowerShell y JScript.
- 2. **Desofuscar** manual y automáticamente hasta el código legible.
- 3. **Identificar** descarga, reflective loading e inyección en memoria.
- 4. **Aprovechar** ScriptBlock Logging, AMSI y ETW para el análisis.
- 5. **Extraer** IOCs y mapear a técnicas de ejecución de ATT&CK.

Temas

#	Tema	Por qué importa
1	Por qué scripts: LOLBins y evasión	Sin binario en disco, menos detección
2	Ofuscación en PowerShell	Base64, formato, alias, IEX
3	Reflective loading e inyección	Ejecuta payloads en memoria
4	JScript/HTA/VBScript (WSH)	Vectores de entrega clásicos
5	Logging y AMSI	Defensas que ayudan al analista
6	Desofuscación segura	Sin ejecutar el código malicioso
7	Extracción de IOCs y TTPs	Detección posterior

Definiciones y características

- **IEX (Invoke-Expression):** ejecuta una cadena como código. Característica clave: **detonador** de payloads ofuscados.
- **Reflective loading:** cargar un ensamblado/PE en memoria sin tocar disco. Característica clave: **fileless**.
- **AMSI:** interfaz antimalware que inspecciona scripts en runtime. Característica clave: **desofusca para el defensor**.

- **ScriptBlock Logging:** registro de bloques de PowerShell ejecutados. Característica clave: **forense de scripts**.
- **HTA:** aplicación HTML ejecutada por mshta. Característica clave: **entrega con confianza del SO**.

Herramientas y preparación

- **CyberChef** y **PowerShell ISE/VSCode** para desofuscar (sin `Invoke`).
- **AMSIContentRetrieval**, **PSDecode**, **box-js** (JScript), **js-beautify**.
- Logs: **Event Viewer** (Microsoft-Windows-PowerShell/Operational), **Sysmon**.
- **REMnux** para las utilidades de JavaScript.

⚠ **Nota ética y de seguridad:** desofusca **sin ejecutar** el código. Sustituye `IEX` / `eval` por impresión en pantalla y trabaja en la VM aislada. Ejecutar el script para "ver qué hace" puede detonar la infección real; usa logging y análisis estático primero.

Laboratorio guiado

PowerShell ofuscado:

1. Identifica las capas: `-enc` (base64 UTF-16LE), `-join`, `[char]`, `(backticks)`, `Replace`, `Format`.
2. Decodifica el base64 con CyberChef (`From Base64` + `Decode text UTF-16LE`).
3. Sustituye el `IEX` final por `Write-Output` para revelar la siguiente capa **sin ejecutarla**. Repite hasta el código limpio.
4. Localiza la lógica real: `Net.WebClient.DownloadString`, `Reflection.Assembly`, shellcode en `[Byte[]]`, `VirtualAlloc / CreateThread` (inyección).
5. Correlaciona con **ScriptBlock Logging**: activa el registro, y en una ejecución controlada previa (o en logs provistos) recupera el bloque desofuscado que Windows ya registró.

JScript/HTA:

6. Beautifica con `js-beautify` y analiza con `box-js` (que emula WSH y registra las llamadas) para capturar URLs y comandos sin ejecutarlo de verdad.
7. Extrae los IOCs (URLs, comandos, hashes) y mapea a ATT&CK (`T1059.001` PowerShell, `T1059.007` JavaScript).

Ejercicios

1. Desofusca un one-liner de PowerShell con 3 capas y documenta cada una.
2. Explica cómo AMSI y ScriptBlock Logging ayudan al analista.
3. Reconoce un patrón de inyección de shellcode en PowerShell.
4. Usa `box-js` sobre un JScript y extrae la URL de segunda etapa.
5. Convierte un `IEX` en `Write-Output` y justifica por qué es más seguro.
6. Redacta una detección basada en el Event ID 4104.

Reto verificable

Desofusca un script malicioso (PowerShell o JScript) hasta su forma legible y entrega la **cadena de ejecución** con la URL/payload de segunda etapa y los IOCs, sin haberlo ejecutado. **Criterio de aceptación:** muestras el

código final desofuscado y explicas cada capa de ofuscación deshecha, con al menos un IOC (URL/comando) verificado en el lab vía INetSim.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Ejecutar el script "para ver"	Detona la infección; desofusca estáticamente o con box-js
Base64 da basura	Es UTF-16LE; ajusta el decode en CyberChef
Sin ScriptBlock logs	No estaba activado; habilítalo por GPO antes de detonar en lab
box-js no resuelve la URL	Emulación incompleta; combina con lectura manual
Perder capas de ofuscación	Ve capa por capa reemplazando el ejecutor por print

Preguntas frecuentes

 **¿Por qué tanto malware en PowerShell?** Está firmado y preinstalado (LOLBin), ejecuta en memoria y ofusca fácilmente, reduciendo la detección basada en archivos.

 **¿AMSI lo detiene todo?** No; existen técnicas de bypass. Aun así, AMSI y el ScriptBlock Logging entregan al defensor el contenido desofuscado, muy útil para el análisis.

 **¿Puedo confiar en box-js?** Es una gran ayuda para JScript, pero emula un subconjunto; complementa siempre con lectura manual del código.

Referencias

- Monnappa K. A., *Learning Malware Analysis*.
- Microsoft — About Script Block Logging.
https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_logging_windows
- box-js. <https://github.com/CapacitorSet/box-js>
- MITRE ATT&CK — Command and Scripting Interpreter (T1059).
<https://attack.mitre.org/techniques/T1059/>

Siguiente clase

Clase 154 - Malware en Linux